RSA (Rivest–Shamir–Adleman)

1. Overview

RSA (**Rivest–Shamir–Adleman, 1977/1978**) is the best-known and most widely deployed **asymmetric (public-key)** cryptosystem.

Unlike AES, RSA uses a **pair** of mathematically related keys:

- **Public key** — used for encryption and signature verification.
- **Private key** — used for decryption and signature creation.

This allows secure communication and authentication without ever sharing a secret key over an insecure channel.

RSA at a Glance

Property	Detail
Type	Asymmetric (public-key) cryptosystem
Basis	Exponentiation in modular arithmetic; security rests on the difficulty of factoring large composite numbers
Common key sizes	2048-bit (current minimum standard), 3072-bit, 4096-bit
Uses	Encryption, key exchange, digital signatures

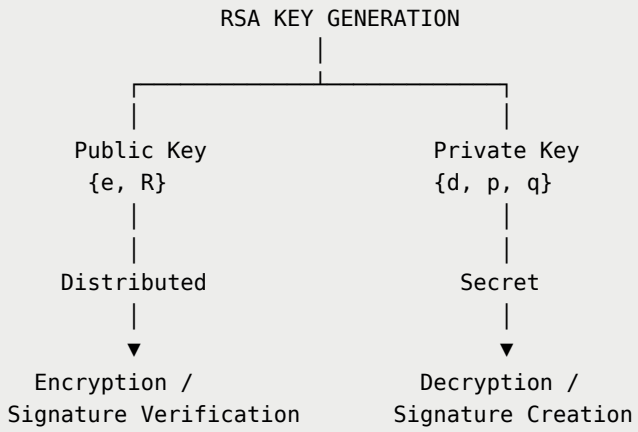
2. Key Generation

RSA key generation follows these steps:

1. Select two large, distinct random primes, p and q (typically several hundred digits each in modern use).
2. Compute the modulus $R = p \times q$.
3. Compute Euler's totient $\Phi(R) = (p - 1)(q - 1)$.
4. Choose a public exponent e such that $1 < e < \Phi(R)$ and $\gcd(e, \Phi(R)) = 1$. A small, fixed value like $e = 65537$ is commonly used in practice for efficiency.
5. Compute the private exponent d satisfying $e \cdot d \equiv 1 \pmod{\Phi(R)}$ (via the extended Euclidean algorithm).

6. **Public key:** `{e, R}` — distributed freely.
7. **Private key:** `{d, p, q}` — kept secret.

Key Relationship



---

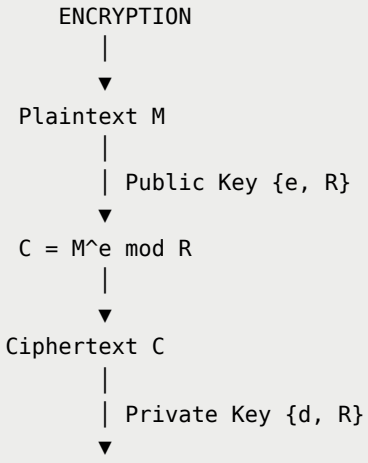
3. Encryption and Decryption

RSA encryption and decryption use modular exponentiation:

```
Encryption:
C = M^e mod R

Decryption:
M = C^d mod R
```

Simplified Flow



```
M = C^d mod R
      |
      ▼
Plaintext M
```

Why It Works

This works because of the underlying number-theoretic identity:

> For `R = pq` with distinct primes `p, q`, `x^Φ(R) ≡ 1 (mod R)` for any `x` not divisible by `p` or `q`.

This guarantees that raising to the `e`-th power and then the `d`-th power returns the original message.

---

4. Why It's Secure — The Hard Problem

RSA's security rests entirely on the **integer factorization problem**.

Given the public modulus `R`, it is computationally infeasible with current classical computers and a sufficiently large `R` to recover the two prime factors `p` and `q`.

Without those factors:

```
Public Modulus R
       |
       ▼
   Factor R = p × q
       |
       ▼
   Compute Φ(R)
       |
       ▼
   Derive private exponent d
```

The best known classical factorization algorithms scale sub-exponentially but still require infeasible amounts of computation for sufficiently large moduli (2048+ bits).

Important Nuance

> Knowing the public key and the algorithm's full public description does **not** help an attacker derive the private key.

This asymmetry — easy to compute in one direction, but infeasible to reverse without the trapdoor — is exactly what makes public-key cryptography possible.

5. Performance Optimizations

RSA's raw modular exponentiation is computationally expensive compared to symmetric ciphers. Several optimizations are therefore standard in real implementations:

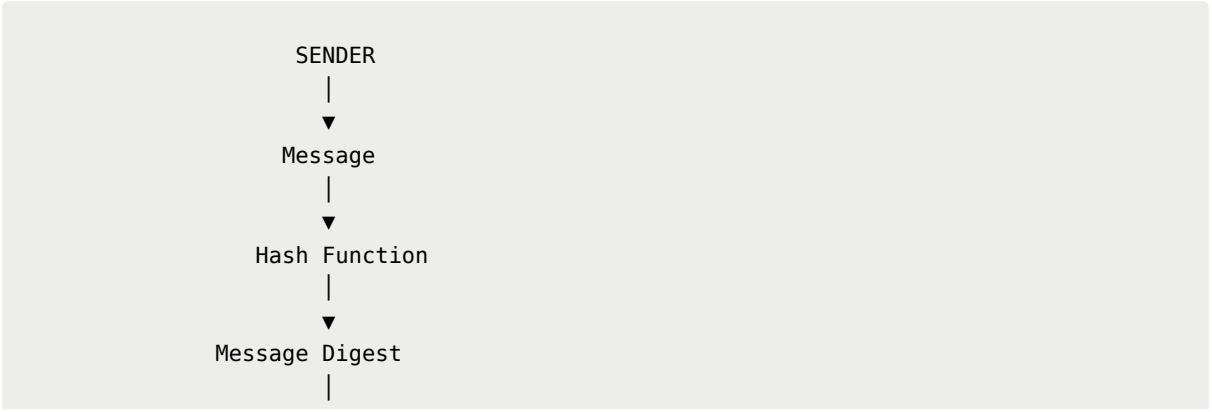
Optimization	Purpose
Chinese Remainder Theorem (CRT)	Decryption can be performed as two smaller computations modulo `p` and `q` separately, then recombined. This yields a significant speedup because arithmetic complexity grows non-linearly with operand size.
Small public exponent	A value such as `e = 65537` makes encryption/signature verification fast because it requires far fewer modular multiplications than a large or random exponent.
Multi-precision arithmetic libraries	RSA numbers vastly exceed native CPU word sizes, so dedicated big-number libraries handle the required arithmetic.

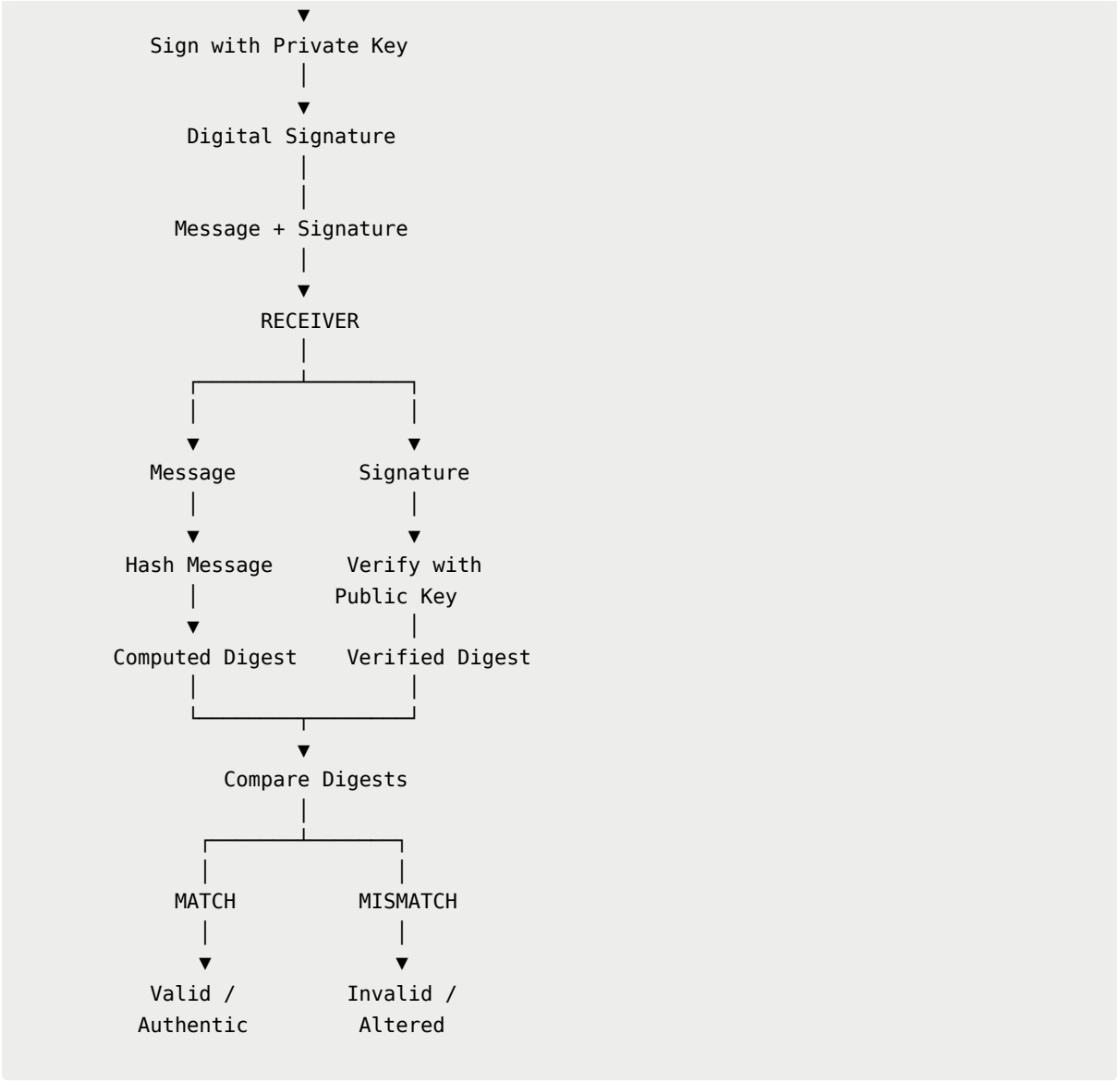
6. Digital Signatures with RSA

RSA can also be used for **digital signatures**.

The signer uses their **private key** to create a signature over a message digest. Anyone can verify the signature using the signer's **public key** and compare the recovered/verified digest against an independently computed hash of the message.

Simplified Signature Flow





RSA digital signatures provide:

- **Authentication**
- **Integrity**
- **Non-repudiation**

---

7. Known Weaknesses and Practical Pitfalls

1. Key Size Matters

Factoring feasibility scales dramatically with modulus size.

> Keys below **2048 bits** are now considered inadequate for new systems.

2. Padding Is Essential

Raw (**"textbook"**) RSA without proper padding is insecure.

Examples of appropriate padding schemes include:

- **OAEP** for encryption
- **PSS** for signatures

Without proper padding, RSA is vulnerable to various structural attacks, including predictable plaintext patterns and chosen-ciphertext attacks.

> **Real-world implementations must never omit padding.**

3. Poor Random Prime Generation

Weak or predictable primes — or primes shared across multiple keys because of flawed random number generators — have historically led to real-world key recovery.

4. Quantum Threat

Shor's algorithm, if run on a sufficiently large fault-tolerant quantum computer, would efficiently factor RSA moduli and break the scheme entirely.

This is a central motivation behind the development of **post-quantum cryptographic alternatives**.

---

8. Typical Use Cases

RSA is commonly used for:

- **TLS/SSL certificate signatures** and historically for key exchange
- **Digital signatures** for code signing, software distribution, and legal e-signatures
- **Secure email** — S/MIME and PGP key exchange
- **Hybrid encryption schemes** — RSA encrypts a symmetric session key, which then encrypts the bulk data using a fast cipher such as AES

Hybrid Encryption Concept

```
                RSA
                 |
```

```
          ▼
Encrypt Session Key
          |
          ▼
Symmetric Session Key
          |
          ▼
         AES
          |
          ▼
 Bulk Data Encryption
```

---

9. Summary

RSA remains one of the most important and widely deployed public-key algorithms, valued for its conceptual simplicity and long track record.

Its practical security depends heavily on:

1. **Adequate key sizes**
2. **Correct padding schemes**
3. **Sound randomness in key generation**
4. **Proper implementation and key management**

Its long-term future is shaped by the ongoing transition toward **post-quantum alternatives** as quantum computing matures.

> **Bottom line:** RSA's security does not come simply from keeping the algorithm secret. The algorithm and public key are designed to be public; security depends on the difficulty of the underlying mathematical problem, sufficiently large keys, correct padding, strong randomness, and protection of the private key.